

Incendio Forestal

Pensamiento Computacional

Facultad de Ciencias Exactas y Naturales, UBA

Segundo Cuatrimestre 2026

REPETIR, REPETIR, REPETIR

Formas de repetir

- En las actividades previas nos encontramos con frecuencia con la necesidad de repetir una acción muchas veces.

Formas de repetir

- En las actividades previas nos encontramos con frecuencia con la necesidad de repetir una acción muchas veces.
- Para esto usamos la sentencia **while**.

Formas de repetir

- En las actividades previas nos encontramos con frecuencia con la necesidad de repetir una acción muchas veces.
- Para esto usamos la sentencia **while**.

Algunos ejemplos:

```
album = [0]*tot_figus
while sum(album)<len(album):
    figu = random.randint(0,tot_figus)
    album[figu] = 1
    cant_figus = cant_figus + 1
```

Formas de repetir

- En las actividades previas nos encontramos con frecuencia con la necesidad de repetir una acción muchas veces.
- Para esto usamos la sentencia **while**.

Algunos ejemplos:

```
album = [0]*tot_figus
while sum(album)<len(album):
    figu = random.randint(0,tot_figus)
    album[figu] = 1
    cant_figus = cant_figus + 1
```

```
i = 0
while i<repeticiones:
    cant = llenar_album(tot_figus)
    cantidades[i] = cant
    i = i+1
```

Formas de repetir

- En las actividades previas nos encontramos con frecuencia con la necesidad de repetir una acción muchas veces.
- Para esto usamos la sentencia **while**.

Algunos ejemplos:

```
album = [0]*tot_figus
while sum(album)<len(album):
    figu = random.randint(0,tot_figus)
    album[figu] = 1
    cant_figus = cant_figus + 1
```

```
i = 0
while i<repeticiones:
    cant = llenar_album(tot_figus)
    cantidades[i] = cant
    i = i+1
```

```
puntajes = [0]*cant_jugadores
j = 0
while j < cant_jugadores:
    tirada = tirar_cubilete()
    p = calcular_puntaje(tirada)
    puntajes[j] = p
    j = j+1
```

Formas de repetir

- En las actividades previas nos encontramos con frecuencia con la necesidad de repetir una acción muchas veces.
- Para esto usamos la sentencia **while**.

Algunos ejemplos:

```
album = [0]*tot_figus
while sum(album)<len(album):
    figu = random.randint(0,tot_figus)
    album[figu] = 1
    cant_figus = cant_figus + 1
```

```
i = 0
while i<repeticiones:
    cant = llenar_album(tot_figus)
    cantidades[i] = cant
    i = i+1
```

```
puntajes = [0]*cant_jugadores
j = 0
while j < cant_jugadores:
    tirada = tirar_cubilete()
    p = calcular_puntaje(tirada)
    puntajes[j] = p
    j = j+1
```

```
fin = False
while not fin:
    puntajes = jugar_ronda(cant_jug)
    acumular(acum_puntajes,puntajes)
    fin = hay_ganador(acum_puntajes)
```

Dos clases de repetición

Tipos de ciclos

Nos hemos encontrado con dos tipos **estructuralmente** distintos de ciclos. Se diferencian en la condición de parada del ciclo. ¿Cuáles vimos?

Dos clases de repetición

Tipos de ciclos

Nos hemos encontrado con dos tipos **estructuralmente** distintos de ciclos. Se diferencian en la condición de parada del ciclo. ¿Cuáles vimos?

- **Ciclos Indefinidos**: Cuando **no** sabemos cuántas iteraciones habrá.

Dos clases de repetición

Tipos de ciclos

Nos hemos encontrado con dos tipos **estructuralmente** distintos de ciclos. Se diferencian en la condición de parada del ciclo. ¿Cuáles vimos?

- **Ciclos Indefinidos**: Cuando **no** sabemos cuántas iteraciones habrá.
Ejemplos:
 - al llenar el álbum: no sabemos cuántas figs necesitaremos comprar.

Dos clases de repetición

Tipos de ciclos

Nos hemos encontrado con dos tipos **estructuralmente** distintos de ciclos. Se diferencian en la condición de parada del ciclo. ¿Cuáles vimos?

- **Ciclos Indefinidos:** Cuando **no** sabemos cuántas iteraciones habrá.

Ejemplos:

- al llenar el álbum: no sabemos cuántas figs necesitaremos comprar.
- al jugar un partido de diez mil: no sabemos cuántas rondas habrá que jugar.

Dos clases de repetición

Tipos de ciclos

Nos hemos encontrado con dos tipos **estructuralmente** distintos de ciclos. Se diferencian en la condición de parada del ciclo. ¿Cuáles vimos?

- **Ciclos Indefinidos**: Cuando **no** sabemos cuántas iteraciones habrá.
Ejemplos:
 - al llenar el álbum: no sabemos cuántas figs necesitaremos comprar.
 - al jugar un partido de diez mil: no sabemos cuántas rondas habrá que jugar.
- **Ciclos Definidos**: Cuando **sabemos de antemano** cuántas iteraciones habrá.

Dos clases de repetición

Tipos de ciclos

Nos hemos encontrado con dos tipos **estructuralmente** distintos de ciclos. Se diferencian en la condición de parada del ciclo. ¿Cuáles vimos?

- **Ciclos Indefinidos**: Cuando **no** sabemos cuántas iteraciones habrá. Ejemplos:
 - al llenar el álbum: no sabemos cuántas figs necesitaremos comprar.
 - al jugar un partido de diez mil: no sabemos cuántas rondas habrá que jugar.
- **Ciclos Definidos**: Cuando **sabemos de antemano** cuántas iteraciones habrá. Ejemplos:

Dos clases de repetición

Tipos de ciclos

Nos hemos encontrado con dos tipos **estructuralmente** distintos de ciclos. Se diferencian en la condición de parada del ciclo. ¿Cuáles vimos?

- **Ciclos Indefinidos**: Cuando **no** sabemos cuántas iteraciones habrá. Ejemplos:
 - al llenar el álbum: no sabemos cuántas figs necesitaremos comprar.
 - al jugar un partido de diez mil: no sabemos cuántas rondas habrá que jugar.
- **Ciclos Definidos**: Cuando **sabemos de antemano** cuántas iteraciones habrá. Ejemplos:
 - al simular muchos llenados de álbum, sabemos cuántas veces vamos a simular.

Dos clases de repetición

Tipos de ciclos

Nos hemos encontrado con dos tipos **estructuralmente** distintos de ciclos. Se diferencian en la condición de parada del ciclo. ¿Cuáles vimos?

- **Ciclos Indefinidos**: Cuando **no** sabemos cuántas iteraciones habrá. Ejemplos:
 - al llenar el álbum: no sabemos cuántas figs necesitaremos comprar.
 - al jugar un partido de diez mil: no sabemos cuántas rondas habrá que jugar.
- **Ciclos Definidos**: Cuando **sabemos de antemano** cuántas iteraciones habrá. Ejemplos:
 - al simular muchos llenados de álbum, sabemos cuántas veces vamos a simular.
 - al jugar una ronda de diez mil: sabemos cuántos jugadores tienen que tirar los dados.

Cuando sabemos la cantidad de iteraciones

Cuando sabemos la cantidad de iteraciones a realizar (porque es un número fijo o porque está almacenado en una variable a la que podemos acceder), tenemos una estructura que se repite:

```
i = 0
while i < TOTAL:
    ...
    i = i+1
```

- Tenemos que inicializar un **contador** (en el ejemplo: `i`).

Cuando sabemos la cantidad de iteraciones

Cuando sabemos la cantidad de iteraciones a realizar (porque es un número fijo o porque está almacenado en una variable a la que podemos acceder), tenemos una estructura que se repite:

```
i = 0
while i < TOTAL:
    ...
    i = i+1
```

- Tenemos que inicializar un **contador** (en el ejemplo: `i`).
- La condición del `while` tiene que ser *no pasarse de la cantidad de iteraciones* (`i < TOTAL`).

Cuando sabemos la cantidad de iteraciones

Cuando sabemos la cantidad de iteraciones a realizar (porque es un número fijo o porque está almacenado en una variable a la que podemos acceder), tenemos una estructura que se repite:

```
i = 0
while i < TOTAL:
    ...
    i = i+1
```

- Tenemos que inicializar un **contador** (en el ejemplo: `i`).
- La condición del `while` tiene que ser *no pasarse de la cantidad de iteraciones* (`i < TOTAL`).
- Luego de las operaciones correspondientes al problema, tenemos que **aumentar el contador**: `i = i+1`.

Cuando sabemos la cantidad de iteraciones

Cuando sabemos la cantidad de iteraciones a realizar (porque es un número fijo o porque está almacenado en una variable a la que podemos acceder), tenemos una estructura que se repite:

```
i = 0
while i < TOTAL:
    ...
    i = i+1
```

- Tenemos que inicializar un **contador** (en el ejemplo: `i`).
- La condición del `while` tiene que ser *no pasarse de la cantidad de iteraciones* (`i < TOTAL`).
- Luego de las operaciones correspondientes al problema, tenemos que **aumentar el contador**: `i = i+1`.
- El **nombre** del contador **no importa**: por convención solemos usar `i` porque es corto y con frecuencia representa un *índice*. Pero puede ser cualquier otro nombre de variable válido (como `contador`, `cant_rondas`, etc.)

FOR

Ciclos definidos: `for`

Cuando repetimos una cantidad prefijada de veces, tenemos una sintaxis simplificada, usando la sentencia **for**:

Ejemplo:

```
for i in range(cant_repeticiones):  
    ...
```

Es equivalente al código:

```
i = 0  
while i < cant_repeticiones:  
    ...  
    i = i+1
```

Ciclos definidos: `for`

Cuando repetimos una cantidad prefijada de veces, tenemos una sintaxis simplificada, usando la sentencia **for**:

Ejemplo:

```
for i in range(cant_repeticiones):  
    ...
```

Es equivalente al código:

```
i = 0  
while i < cant_repeticiones:  
    ...  
    i = i+1
```

Importante

- La variable `i` **la define el propio `for`**. No hay que definirla aparte.
- La actualización `i = i+1` **también la hace el `for` automáticamente** (ya veremos cómo).

range

¿Pero... qué hace exactamente `range`?

- `range` genera un *rango* (una secuencia) de valores.

range

¿Pero... qué hace exactamente `range`?

- `range` genera un *rango* (una secuencia) de valores.
- `range(b)`: números enteros **mayores o iguales** que 0 y **menores** que `b`.

range

¿Pero... qué hace exactamente `range`?

- `range` genera un *rango* (una secuencia) de valores.
- `range(b)`: números enteros **mayores o iguales** que 0 y **menores** que `b`.
- `range(a, b)`: números enteros **mayores o iguales** que `a` y **menores** que `b`.

range

¿Pero... qué hace exactamente `range`?

- `range` genera un *rango* (una secuencia) de valores.
- `range(b)`: números enteros **mayores o iguales** que 0 y **menores** que `b`.
- `range(a, b)`: números enteros **mayores o iguales** que `a` y **menores** que `b`.
- `range(a, b, k)`: números enteros desde `a` hasta `b` (sin incluir a `b`), saltando de `a` a `k` unidades (el incremento puede ser negativo para ir hacia *atrás*).

range

¿Pero... qué hace exactamente `range`?

- `range` genera un *rango* (una secuencia) de valores.
- `range(b)`: números enteros **mayores o iguales** que 0 y **menores** que `b`.
- `range(a, b)`: números enteros **mayores o iguales** que `a` y **menores** que `b`.
- `range(a, b, k)`: números enteros desde `a` hasta `b` (sin incluir a `b`), saltando de `a` a `k` unidades (el incremento puede ser negativo para ir hacia *atrás*).

range

¿Pero... qué hace exactamente `range`?

- `range` genera un *rango* (una secuencia) de valores.
- `range(b)`: números enteros **mayores o iguales** que 0 y **menores** que `b`.
- `range(a, b)`: números enteros **mayores o iguales** que `a` y **menores** que `b`.
- `range(a, b, k)`: números enteros desde `a` hasta `b` (sin incluir a `b`), saltando de `a` `k` unidades (el incremento puede ser negativo para ir hacia *atrás*).

Interesante

Si quisiéramos generar una lista de números enteros en cierto rango (tan grande como uno quiera), podemos usar `range`:

```
my_range = range(1,1000)
print(my_range)
my_list = list(my_range)
print(my_list)
```

Algunos ejemplos

```
for i in range(3, 6):  
    print(i)
```

¿Qué imprime?

Algunos ejemplos

```
for i in range(3, 6):  
    print(i)
```

¿Qué imprime?
Los números: 3,4,5.

```
for j in range(3, 8, 2):  
    print(j)
```

¿Qué imprime?

Algunos ejemplos

```
for i in range(3,6):  
    print(i)
```

¿Qué imprime?
Los números: 3,4,5.

```
for j in range(3,8,2):  
    print(j)
```

¿Qué imprime?
Los números: 3,5,7.

```
for pepe in range(10,7,-1):  
    print(pepe)
```

¿Qué imprime?

Algunos ejemplos

```
for i in range(3,6):  
    print(i)
```

¿Qué imprime?
Los números: 3,4,5.

```
for j in range(3,8,2):  
    print(j)
```

¿Qué imprime?
Los números: 3,5,7.

```
for pepe in range(10,7,-1):  
    print(pepe)
```

¿Qué imprime?
Los números 10,9,8.

Más sobre `range`

En la práctica, podemos pensar que `range` *genera* una **secuencia** de valores. De hecho, también podés usar **for** directamente sobre una lista:

```
puntajes = [150, 500, 1100, 350]
for puntos in puntajes:
    print(puntos)
```

Más sobre `range`

En la práctica, podemos pensar que `range` *genera* una **secuencia** de valores. De hecho, también podés usar **for** directamente sobre una lista:

```
puntajes = [150, 500, 1100, 350]
for puntos in puntajes:
    print (puntos)
```

- Imprime los valores de `puntajes`, uno a uno.
- Es muy útil cuando necesitamos los elementos y no nos interesa su posición.
- Hay muchas estructuras que permiten ser iteradas de una forma similar.

Pregunta maliciosa

¿Qué imprime el siguiente código?

```
for i in range(4):  
    print(i)  
    i = 5
```

Pregunta maliciosa

¿Qué imprime el siguiente código?

```
for i in range(4):  
    print(i)  
    i = 5
```

Los números: 0,1,2,3.

- Comienza en $i=0$ y lo imprime.

Pregunta maliciosa

¿Qué imprime el siguiente código?

```
for i in range(4):  
    print(i)  
    i = 5
```

Los números: 0,1,2,3.

- Comienza en `i=0` y lo imprime.
- Pone `i=5` y allí termina la primera vuelta.

Pregunta maliciosa

¿Qué imprime el siguiente código?

```
for i in range(4):  
    print(i)  
    i = 5
```

Los números: 0,1,2,3.

- Comienza en `i=0` y lo imprime.
- Pone `i=5` y allí termina la primera vuelta.
- Luego, el `for` pasa el valor de `i` al siguiente elemento del rango, `i=1`, lo imprime.

Pregunta maliciosa

¿Qué imprime el siguiente código?

```
for i in range(4):  
    print(i)  
    i = 5
```

Los números: 0,1,2,3.

- Comienza en `i=0` y lo imprime.
- Pone `i=5` y allí termina la primera vuelta.
- Luego, el `for` pasa el valor de `i` al siguiente elemento del rango, `i=1`, lo imprime.
- Y así sigue.

Pregunta maliciosa

¿Qué imprime el siguiente código?

```
for i in range(4):  
    print(i)  
    i = 5
```

Los números: 0,1,2,3.

- Comienza en `i=0` y lo imprime.
- Pone `i=5` y allí termina la primera vuelta.
- Luego, el `for` pasa el valor de `i` al siguiente elemento del rango, `i=1`, lo imprime.
- Y así sigue.

Pregunta maliciosa

¿Qué imprime el siguiente código?

```
for i in range(4):  
    print(i)  
    i = 5
```

Los números: 0,1,2,3.

- Comienza en $i=0$ y lo imprime.
- Pone $i=5$ y allí termina la primera vuelta.
- Luego, el `for` pasa el valor de i al siguiente elemento del rango, $i=1$, lo imprime.
- Y así sigue.

Importante

La estructura `range` va generando la secuencia de valores en cada iteración. Cada *vuelta* del `for` consigue el siguiente elementos hasta que no hay más.

ARRAYS

Arrays

En una ronda del juego de Diez Mil teníamos la siguiente dinámica:

- Comienza el juego con una lista de puntajes generales todos iguales a 0.

Arrays

En una ronda del juego de Diez Mil teníamos la siguiente dinámica:

- Comienza el juego con una lista de puntajes generales todos iguales a 0.
- Los jugadores juegan una ronda: cada cual tira los dados y suma sus puntos. Se genera una lista de puntajes de la ronda.

Arrays

En una ronda del juego de Diez Mil teníamos la siguiente dinámica:

- Comienza el juego con una lista de puntajes generales todos iguales a 0.
- Los jugadores juegan una ronda: cada cual tira los dados y suma sus puntos. Se genera una lista de puntajes de la ronda.
- Es necesario **acumular** los puntos, es decir: sumar los puntajes de la ronda sobre la lista de puntajes generales.

Arrays

En una ronda del juego de Diez Mil teníamos la siguiente dinámica:

- Comienza el juego con una lista de puntajes generales todos iguales a 0.
- Los jugadores juegan una ronda: cada cual tira los dados y suma sus puntos. Se genera una lista de puntajes de la ronda.
- Es necesario **acumular** los puntos, es decir: sumar los puntajes de la ronda sobre la lista de puntajes generales.

Podría hacer algo como:

```
n = len(puntajes)
for i in range(n):
    puntajes_general[i] = puntajes_general[i] + puntajes[i]
```

Arrays

En una ronda del juego de Diez Mil teníamos la siguiente dinámica:

- Comienza el juego con una lista de puntajes generales todos iguales a 0.
- Los jugadores juegan una ronda: cada cual tira los dados y suma sus puntos. Se genera una lista de puntajes de la ronda.
- Es necesario **acumular** los puntos, es decir: sumar los puntajes de la ronda sobre la lista de puntajes generales.

Podría hacer algo como:

```
n = len(puntajes)
for i in range(n):
    puntajes_general[i] = puntajes_general[i] + puntajes[i]
```

Esta operación es bastante común:

Dadas dos listas de números (de igual longitud), quiero sumarlas **casillero a casillero**.

Arrays

En una ronda del juego de Diez Mil teníamos la siguiente dinámica:

- Comienza el juego con una lista de puntajes generales todos iguales a 0.
- Los jugadores juegan una ronda: cada cual tira los dados y suma sus puntos. Se genera una lista de puntajes de la ronda.
- Es necesario **acumular** los puntos, es decir: sumar los puntajes de la ronda sobre la lista de puntajes generales.

Podría hacer algo como:

```
n = len(puntajes)
for i in range(n):
    puntajes_general[i] = puntajes_general[i] + puntajes[i]
```

Esta operación es bastante común:

Dadas dos listas de números (de igual longitud), quiero sumarlas **casillero a casillero**. Este tipo de funcionalidades (matemática con *listas*) se encuentra en el módulo `numpy`.

Numerical Python - numpy

Importamos el módulo

```
import numpy as np
```

Numerical Python - numpy

Importamos el módulo

```
import numpy as np
```

- Podríamos usar `import numpy.`

Numerical Python - numpy

Importamos el módulo

```
import numpy as np
```

- Podríamos usar `import numpy`.
- Agregar `as np` le pone un *alias* numpy: np.

Numerical Python - numpy

Importamos el módulo

```
import numpy as np
```

- Podríamos usar `import numpy`.
- Agregar `as np` le pone un *alias* numpy: np.
- Es una convención de la comunidad, van a encontrar que todos los ejemplos que encuentren utilizan ese alias al momento de importar.

Numerical Python - *numpy*

Importamos el módulo

```
import numpy as np
```

- Podríamos usar `import numpy`.
- Agregar `as np` le pone un *alias* `numpy: np`.
- Es una convención de la comunidad, van a encontrar que todos los ejemplos que encuentren utilizan ese alias al momento de importar.

Para crear un `array` hacemos, por ejemplo:

```
puntajes = np.array([0,0,0,0])
```

Numerical Python - numpy

Importamos el módulo

```
import numpy as np
```

- Podríamos usar `import numpy`.
- Agregar `as np` le pone un *alias* `numpy: np`.
- Es una convención de la comunidad, van a encontrar que todos los ejemplos que encuentren utilizan ese alias al momento de importar.

Para crear un `array` hacemos, por ejemplo:

```
puntajes = np.array([0,0,0,0])
```

Esto crea un `array` de longitud 4 (con cuatro elementos), todo lleno de ceros. Un `array`, en principio, es *parecido* a una lista: una colección de valores.

Numerical Python - numpy

Importamos el módulo

```
import numpy as np
```

- Podríamos usar `import numpy`.
- Agregar `as np` le pone un *alias* `numpy: np`.
- Es una convención de la comunidad, van a encontrar que todos los ejemplos que encuentren utilizan ese alias al momento de importar.

Para crear un `array` hacemos, por ejemplo:

```
puntajes = np.array([0,0,0,0])
```

Esto crea un `array` de longitud 4 (con cuatro elementos), todo lleno de ceros. Un `array`, en principio, es *parecido* a una lista: una colección de valores. Pero permite hacer operaciones matemáticas directamente:

```
puntajes_general = np.array([0,0,0,0])  
puntajes = np.array([150,300,250,1000])  
puntajes_general = puntajes_general + puntajes
```

realiza la suma **casillero a casillero**.

Más sobre arrays

Se pueden crear con funciones especiales:

- `np.zeros(4)`: crea un **array de ceros** de longitud 4.
- `np.ones(8)`: crea un **array de unos** de longitud 8.

Más sobre arrays

Se pueden crear con funciones especiales:

- `np.zeros(4)`: crea un **array de ceros** de longitud 4.
- `np.ones(8)`: crea un **array de unos** de longitud 8.

Se **indexan** y **modifican** igual que las listas:

```
import numpy as np
import random

def tirar_cubilete():
    dados = np.zeros(5)
    for i in range(5):
        dados[i] = random.randint(1,6)
    return dados
```

Listas vs. Arrays de NumPy

Ya vimos que ambos **almacenan colecciones** de datos, pero funcionan de manera muy diferente cuando operamos con ellos:

Listas vs. Arrays de NumPy

Ya vimos que ambos **almacenan colecciones** de datos, pero funcionan de manera muy diferente cuando operamos con ellos:

Listas nativas (Python)

```
a = [1, 2, 3]
b = [4, 5, 6]

# Junta las listas (concatena)
print(a + b)
# Resultado: [1, 2, 3, 4, 5, 6]

# Repite la lista
print(a * 2)
# Resultado: [1, 2, 3, 1, 2, 3]
```

Listas vs. Arrays de NumPy

Ya vimos que ambos **almacenan colecciones** de datos, pero funcionan de manera muy diferente cuando operamos con ellos:

Listas nativas (Python)

```
a = [1, 2, 3]
b = [4, 5, 6]

# Junta las listas (concatena)
print(a + b)
# Resultado: [1, 2, 3, 4, 5, 6]

# Repite la lista
print(a * 2)
# Resultado: [1, 2, 3, 1, 2, 3]
```

Arrays de NumPy

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Suma casillero a casillero
print(a + b)
# Resultado: [5, 7, 9]

# Multiplica cada elemento
print(a * 2)
# Resultado: [2, 4, 6]
```

Listas vs. Arrays de NumPy

Ya vimos que ambos **almacenan colecciones** de datos, pero funcionan de manera muy diferente cuando operamos con ellos:

Listas nativas (Python)

```
a = [1, 2, 3]
b = [4, 5, 6]

# Junta las listas (concatena)
print(a + b)
# Resultado: [1, 2, 3, 4, 5, 6]

# Repite la lista
print(a * 2)
# Resultado: [1, 2, 3, 1, 2, 3]
```

Arrays de NumPy

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Suma casillero a casillero
print(a + b)
# Resultado: [5, 7, 9]

# Multiplica cada elemento
print(a * 2)
# Resultado: [2, 4, 6]
```

En resumen

Las **listas** modifican su estructura (crecen). Los **arrays** aplican las operaciones a cada uno de sus elementos.

INCENDIO

Incendio Forestal



 **INCENDIO DE COPAS**

Sistemas complejos

- Sistemas en los cuales el comportamiento **colectivo** es difícil de anticipar a partir del comportamiento **individual** de sus elementos.
- Las reglas a nivel microscópico (de cada elemento) suelen ser bastante simples pero generar propiedades **emergentes** a nivel macroscópico que son **sorprendentes**.

La computadora nos permite programar, simular, explorar y detectar estas cosas.

¿Por qué *incendios de bosques*?

- Fue uno de los primeros modelos (~ 1992), con el de avalanchas en pilas de arena.
- Buscaba modelar la dinámica de un bosque, que consiste en el crecimiento de árboles y la aparición de incendios que los eliminan. Entender por qué los bosques tienen los tamaños que tienen, y se alternan sectores con árboles y sectores sin árboles.

¿Por qué *incendios de bosques*?

- Fue uno de los primeros modelos (~ 1992), con el de avalanchas en pilas de arena.
- Buscaba modelar la dinámica de un bosque, que consiste en el crecimiento de árboles y la aparición de incendios que los eliminan. Entender por qué los bosques tienen los tamaños que tienen, y se alternan sectores con árboles y sectores sin árboles.
- Hay mil variantes, pero ninguna explica ningún bosque conocido. ¿Por qué no nos olvidamos del modelo, entonces?
- Porque se aplica en otros problemas, como epidemias o la difusión de noticias/rumores...

El modelo (primera aproximación)

Modelaremos ciclos de un año donde a cada parte del bosque le suceden cosas.

El modelo (primera aproximación)

Modelaremos ciclos de un año donde a cada parte del bosque le suceden cosas.

Bosque es un segmento lineal, dividido en n celdas, y cada una puede:

- tener un árbol
- no tener nada

El modelo (primera aproximación)

Modelaremos ciclos de un año donde a cada parte del bosque le suceden cosas.

Bosque es un segmento lineal, dividido en n celdas, y cada una puede:

- tener un árbol
- no tener nada

Durante cada año se suceden una serie de **etapas**:

- Brotes
- Caída de rayos
- Propagación de incendios
- Limpieza

Etapas

1. **Brotes:** en cada celda vacía, puede brotar un nuevo árbol (o no).

Etapas

1. **Brotes:** en cada celda vacía, puede brotar un nuevo árbol (o no).
2. **Caída de rayos:** caen rayos en algunas posiciones, incendiando a sus árboles.

Etapas

1. **Brotes:** en cada celda vacía, puede brotar un nuevo árbol (o no).
2. **Caída de rayos:** caen rayos en algunas posiciones, incendiando a sus árboles.
3. **Propagación de incendios:** Cuando un árbol se incendia, propaga el fuego a los árboles de **celdas vecinas**

Etapas

1. **Brotes:** en cada celda vacía, puede brotar un nuevo árbol (o no).
2. **Caída de rayos:** caen rayos en algunas posiciones, incendiando a sus árboles.
3. **Propagación de incendios:** Cuando un árbol se incendia, propaga el fuego a los árboles de **celdas vecinas**, y estos, a su vez, a sus vecinos...

Etapas

1. **Brotes:** en cada celda vacía, puede brotar un nuevo árbol (o no).
2. **Caída de rayos:** caen rayos en algunas posiciones, incendiando a sus árboles.
3. **Propagación de incendios:** Cuando un árbol se incendia, propaga el fuego a los árboles de **celdas vecinas**, y estos, a su vez, a sus vecinos... y así, hasta que todo árbol incendiado incendió a todos sus vecinos y el incendio no puede propagarse más.

Etapas

1. **Brotes:** en cada celda vacía, puede brotar un nuevo árbol (o no).
2. **Caída de rayos:** caen rayos en algunas posiciones, incendiando a sus árboles.
3. **Propagación de incendios:** Cuando un árbol se incendia, propaga el fuego a los árboles de **celdas vecinas**, y estos, a su vez, a sus vecinos... y así, hasta que todo árbol incendiado incendió a todos sus vecinos y el incendio no puede propagarse más.
4. **Limpieza:** los árboles incendiados degradan y dejan la posición vacía, lista para que vuelva a comenzar el ciclo.

Etapas de la dinámica de un bosque - Año 0

Inicio

--	--	--	--	--	--	--

Etapas de la dinámica de un bosque - Año 0

Inicio							
Brotes							

Etapas de la dinámica de un bosque - Año 0

Inicio						
Brotes						
Rayos						

Etapas de la dinámica de un bosque - Año 0

Inicio						
Brotes						
Rayos						
						

Etapas de la dinámica de un bosque - Año 0

Inicio						
Brotes						
Rayos						
						
Propagación						

Etapas de la dinámica de un bosque - Año 0

Inicio						
Brotes						
Rayos						
Propagación						
						
						
Limpieza						

Etapas de la dinámica de un bosque - Año 0

Inicio						
Brotes						
Rayos						
Propagación						
						
Limpieza						

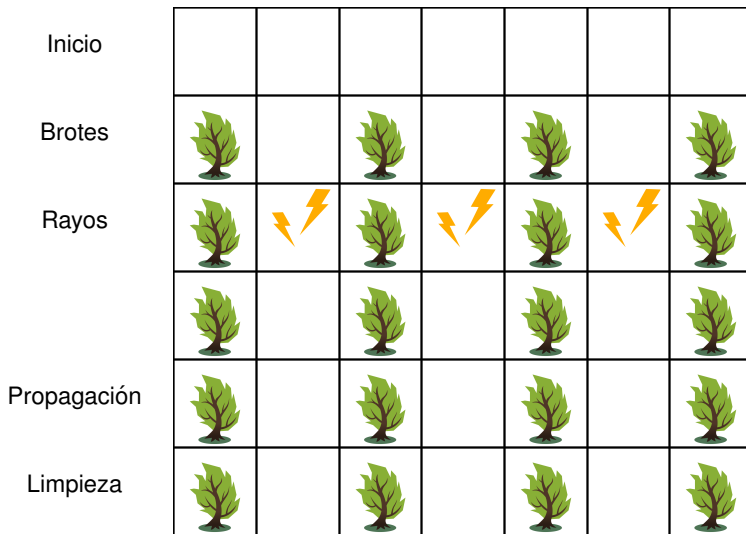
Al final del año: 1 árbol.

Etapas de la dinámica de un bosque - Año 1

Inicio							
Brotes							
Rayos							
Propagación							
							
Limpieza							

Al final del año 1: 0 árboles.

Etapas de la dinámica de un bosque - Año 2



Al final del año 2: 4 árboles.

Etapas de la dinámica de un bosque - Año 3

Inicio							
Brotes							
Rayos							
Propagación							
							
Limpieza							

Al final del año 3: 1 árbol.

Modelo de incendios de bosques

Queremos tomar los ingredientes básicos del problema real y construir un modelo.

Modelo de incendios de bosques

Queremos tomar los ingredientes básicos del problema real y construir un modelo.

- **Bosque:** tenemos n celdas, y en cada una puede crecer un solo árbol.

Modelo de incendios de bosques

Queremos tomar los ingredientes básicos del problema real y construir un modelo.

- **Bosque:** tenemos n celdas, y en cada una puede crecer un solo árbol.
- **Brotes:** brota un árbol en cada celda vacía, con probabilidad p (la calidad del terreno).

Modelo de incendios de bosques

Queremos tomar los ingredientes básicos del problema real y construir un modelo.

- **Bosque:** tenemos n celdas, y en cada una puede crecer un solo árbol.
- **Brotes:** brota un árbol en cada celda vacía, con probabilidad p (la calidad del terreno).
- **Caída de rayos:** cae un rayo en cada celda con probabilidad f . Si la celda tiene un árbol, se prende fuego.

Modelo de incendios de bosques

Queremos tomar los ingredientes básicos del problema real y construir un modelo.

- **Bosque:** tenemos n celdas, y en cada una puede crecer un solo árbol.
- **Brotes:** brota un árbol en cada celda vacía, con probabilidad p (la calidad del terreno).
- **Caída de rayos:** cae un rayo en cada celda con probabilidad f . Si la celda tiene un árbol, se prende fuego.
- **Propagación de incendios:** si un árbol está prendido fuego y tiene al lado un árbol sano, éste también se prende fuego. Se termina cuando no queda ningún árbol sano con un vecino prendido fuego.

Modelo de incendios de bosques

Queremos tomar los ingredientes básicos del problema real y construir un modelo.

- **Bosque:** tenemos n celdas, y en cada una puede crecer un solo árbol.
- **Brotes:** brota un árbol en cada celda vacía, con probabilidad p (la calidad del terreno).
- **Caída de rayos:** cae un rayo en cada celda con probabilidad f . Si la celda tiene un árbol, se prende fuego.
- **Propagación de incendios:** si un árbol está prendido fuego y tiene al lado un árbol sano, éste también se prende fuego. Se termina cuando no queda ningún árbol sano con un vecino prendido fuego.
- **Limpieza:** los árboles quemados se retiran dejando la celda vacía.

Representación

¿Cómo representar esto en la máquina?

- **Bosque:** tomamos un arreglo (`array`) unidimensional de longitud n . Las posiciones van del 0 al $n - 1$.

Representación

¿Cómo representar esto en la máquina?

- **Bosque:** tomamos un arreglo (`array`) unidimensional de longitud n . Las posiciones van del 0 al $n - 1$.
- **Árboles:** representamos las posiciones como:
 - 0 si está vacía,
 - 1 si hay un árbol vivo,
 - 1 si hay un árbol prendido fuego.

Problemas interesantes a resolver:

1. Para un valor fijo de f , ¿cuál es el valor de p que **maximiza** la cantidad de árboles que sobreviven cada año?

Problemas interesantes a resolver:

1. Para un valor fijo de f , ¿cuál es el valor de p que **maximiza** la cantidad de árboles que sobreviven cada año?
2. Si hubiera posiciones con **distinto** valor de p (mayor facilidad de brote), ¿qué pasaría?

Problemas interesantes a resolver:

1. Para un valor fijo de f , ¿cuál es el valor de p que **maximiza** la cantidad de árboles que sobreviven cada año?
2. Si hubiera posiciones con **distinto** valor de p (mayor facilidad de brote), ¿qué pasaría?
3. ¿Y si modelamos un sistema donde la propagación **no sólo** se da entre vecinos?

Problemas interesantes a resolver:

1. Para un valor fijo de f , ¿cuál es el valor de p que **maximiza** la cantidad de árboles que sobreviven cada año?
2. Si hubiera posiciones con **distinto** valor de p (mayor facilidad de brote), ¿qué pasaría?
3. ¿Y si modelamos un sistema donde la propagación **no sólo** se da entre vecinos?
4. ¿Y si el viento hace que la **propagación** no sea siempre igual?

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**?

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**?
Por ejemplo, ¿cómo simulamos la caída de un rayo con probabilidad del 3 %?

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**? Por ejemplo, ¿cómo simulamos la caída de un rayo con probabilidad del 3 %?

Herramientas: Tenemos las funciones `random.randint()` y `random.random()`.

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**? Por ejemplo, ¿cómo simulamos la caída de un rayo con probabilidad del 3%?

Herramientas: Tenemos las funciones `random.randint()` y `random.random()`.

Tiramos un número al azar entre 1 y 100:

```
dado = random.randint(1,100)
```

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**? Por ejemplo, ¿cómo simulamos la caída de un rayo con probabilidad del 3%?

Herramientas: Tenemos las funciones `random.randint()` y `random.random()`.

Tiramos un número al azar entre 1 y 100:

```
dado = random.randint(1,100)
```

¿Cuán probable es sacar un número menor o igual a 3?

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**? Por ejemplo, ¿cómo simulamos la caída de un rayo con probabilidad del 3%?

Herramientas: Tenemos las funciones `random.randint()` y `random.random()`.

Tiramos un número al azar entre 1 y 100:

```
dado = random.randint(1,100)
```

¿Cuán probable es sacar un número menor o igual a 3? 3%

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**? Por ejemplo, ¿cómo simulamos la caída de un rayo con probabilidad del 3%?

Herramientas: Tenemos las funciones `random.randint()` y `random.random()`.

Tiramos un número al azar entre 1 y 100:

```
dado = random.randint(1,100)
```

¿Cuán probable es sacar un número menor o igual a 3? 3%

Entonces podemos hacer algo así:

```
def cae_rayo():  
    cae = False  
    dado = random.randint(1,100)  
    if dado <= 3:  
        cae = True  
    return cae
```

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**? Por ejemplo, ¿cómo simulamos la caída de un rayo con probabilidad del 3%?

Herramientas: Tenemos las funciones `random.randint()` y `random.random()`.

Tiramos un número al azar entre 1 y 100:

```
dado = random.randint(1,100)
```

¿Cuán probable es sacar un número menor o igual a 3? 3%

Entonces podemos hacer algo así:

```
def cae_rayo():  
    cae = False  
    dado = random.randint(1,100)  
    if dado <= 3:  
        cae = True  
    return cae
```

O mucho más corto:

Pregunta

¿Cómo podemos *simular* la ocurrencia de un evento con una **probabilidad prefijada**? Por ejemplo, ¿cómo simulamos la caída de un rayo con probabilidad del 3%?

Herramientas: Tenemos las funciones `random.randint()` y `random.random()`.

Tiramos un número al azar entre 1 y 100:

```
dado = random.randint(1,100)
```

¿Cuán probable es sacar un número menor o igual a 3? 3%

Entonces podemos hacer algo así:

```
def cae_rayo():  
    cae = False  
    dado = random.randint(1,100)  
    if dado <= 3:  
        cae = True  
    return cae
```

O mucho más corto:

```
def cae_rayo():  
    return random.randint(1,100)<=3
```

Con `random.random()`

¿Qué pasa si queremos más detalle, por ejemplo: cae el rayo con probabilidad 2,95 %?

Con `random.random()`

¿Qué pasa si queremos más detalle, por ejemplo: cae el rayo con probabilidad 2,95 %?
`random.randint()` no se presta fácilmente para tanta precisión.

Con `random.random()`

¿Qué pasa si queremos más detalle, por ejemplo: cae el rayo con probabilidad 2,95 %?

`random.randint()` no se presta fácilmente para tanta precisión.

Es mejor usar `random.random()`.

Hay que tener en cuenta que `random.random()` devuelve números entre 0 y 1. Para *traducirlo* a porcentaje lo podemos multiplicar por 100.

Con `random.random()`

¿Qué pasa si queremos más detalle, por ejemplo: cae el rayo con probabilidad 2,95 %?

`random.randint()` no se presta fácilmente para tanta precisión.

Es mejor usar `random.random()`.

Hay que tener en cuenta que `random.random()` devuelve números entre 0 y 1. Para *traducirlo* a porcentaje lo podemos multiplicar por 100.

Entonces, podríamos verificar:

```
100*random.random() <= 2.95
```

Con `random.random()`

¿Qué pasa si queremos más detalle, por ejemplo: cae el rayo con probabilidad 2,95 %?

`random.randint()` no se presta fácilmente para tanta precisión.

Es mejor usar `random.random()`.

Hay que tener en cuenta que `random.random()` devuelve números entre 0 y 1. Para *traducirlo* a porcentaje lo podemos multiplicar por 100.

Entonces, podríamos verificar:

```
100*random.random() <= 2.95
```

o también:

```
random.random() <= 2.95/100
```

Con `random.random()`

¿Qué pasa si queremos más detalle, por ejemplo: cae el rayo con probabilidad 2,95 %?

`random.randint()` no se presta fácilmente para tanta precisión.

Es mejor usar `random.random()`.

Hay que tener en cuenta que `random.random()` devuelve números entre 0 y 1. Para *traducirlo* a porcentaje lo podemos multiplicar por 100.

Entonces, podríamos verificar:

```
100*random.random() <= 2.95
```

o también:

```
random.random() <= 2.95/100
```

es decir:

```
random.random() <= 0.0295
```

Experimentando la dinámica

Pre-requisitos:

Experimentando la dinámica

Pre-requisitos:

- Una computadora donde poder correr `random.random()`.

Experimentando la dinámica

Pre-requisitos:

- Una computadora donde poder correr `random.random()`.
- También pueden usar `https://www.random.org/decimal-fractions/` pidiendo que generen un solo valor.

Experimentando la dinámica

Pre-requisitos:

- Una computadora donde poder correr `random.random()`.
- También pueden usar `https://www.random.org/decimal-fractions/` pidiendo que generen un solo valor.
- Una tarjeta blanca, una verde y una roja.

Experimentando la dinámica

Pre-requisitos:

- Una computadora donde poder correr `random.random()`.
- También pueden usar `https://www.random.org/decimal-fractions/` pidiendo que generen un solo valor.
- Una tarjeta blanca, una verde y una roja.
- Determinar quiénes son tus vecinos.

A simular

A simular

- Todos empiezan siendo celdas vacías.

A simular

- Todos empiezan siendo celdas vacías.
- Todos los que saquen entre 0 y 0.8 se transforman en árboles.

A simular

- Todos empiezan siendo celdas vacías.
- Todos los que saquen entre 0 y 0.8 se transforman en árboles.
- Todos los que saquen entre 0 y 0.3 se prenden fuego.

A simular

- Todos empiezan siendo celdas vacías.
- Todos los que saquen entre 0 y 0.8 se transforman en árboles.
- Todos los que saquen entre 0 y 0.3 se prenden fuego.
- A propagar.

A simular

- Todos empiezan siendo celdas vacías.
- Todos los que saquen entre 0 y 0.8 se transforman en árboles.
- Todos los que saquen entre 0 y 0.3 se prenden fuego.
- A propagar.
- Limpiar.

A simular

- Todos empiezan siendo celdas vacías.
- Todos los que saquen entre 0 y 0.8 se transforman en árboles.
- Todos los que saquen entre 0 y 0.3 se prenden fuego.
- A propagar.
- Limpiar.
- Otra vez.

Esto recién comienza. . .

Terminamos. . . ¡sólo por hoy!